Computationally Verifying a Fundamental Theorem of Directed Graphs

Presented by

- Abdur Rahman Moayed (2025-1-60-015)
- Shafkat Rahman (2025-1-60-001)
- Md. Sadman Khan (2025-1-60-012)
- Sadman Showmik Anik (2025-1-60-055)

Presented to

- Sadia Nur Amin,
- Lecturer, Dept. of Computer Science & Engineering,
- East West University.

Insights into graph theory applications

Our Project's Objective

Build Graphs

Generate large, random directed graphs with up to 5000 vertices using a C program.

Analyze Algorithm

Measure and analyze the algorithm's performance and time complexity.

Verify Proof

Computationally prove that the theorem holds true for these large-scale graphs.

Key Concepts of the Theorem

01 Directed Graph (DiGraph)

A set of vertices (nodes) connected by edges that have a specific direction. It's the perfect structure for modeling one-way relationships.

02 Adjacency Matrix

An n x n grid where matrix[i][j] stores the number of directed edges from vertex i to vertex j. This provides a simple and direct way to represent a graph in code.

03 Out-Degree: "Connections Leaving"

The sum of a vertex's row in the adjacency matrix.

04 In-Degree: "Connections Arriving"

The sum of a vertex's column in the adjacency matrix.

Methodology: Step-by-Step Process

This section outlines a systematic approach, detailing **five essential steps** in our methodology.

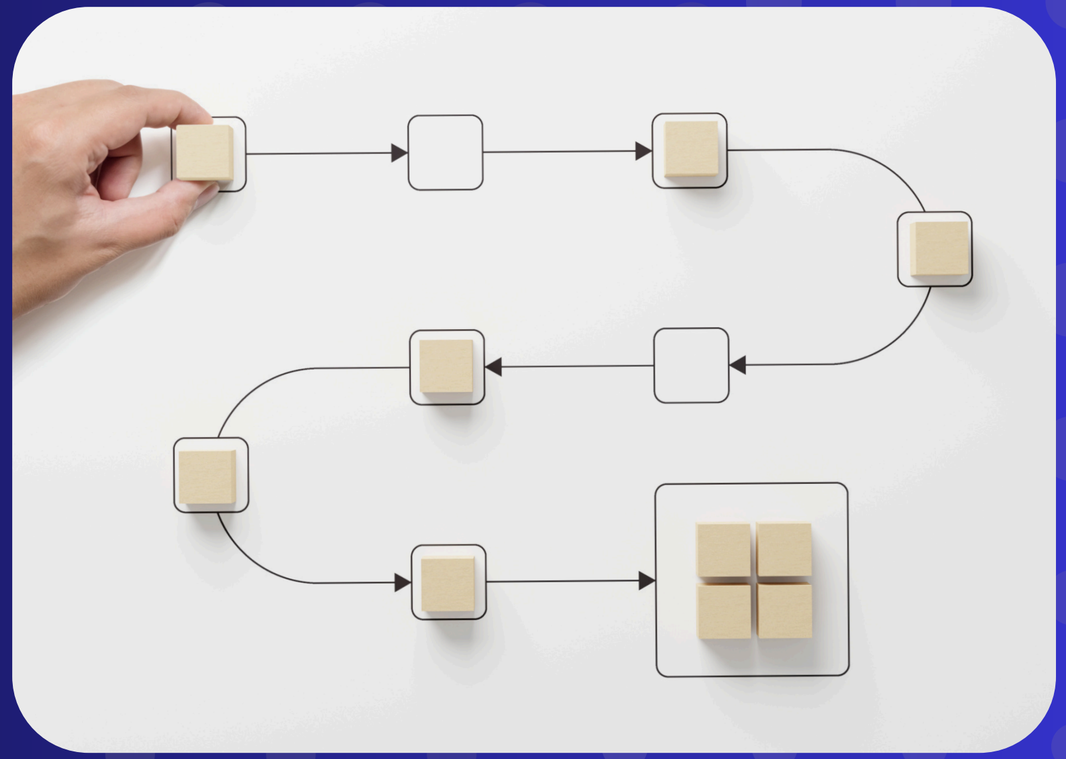
1. Generate Graph
Create an n x n matrix with random edge weights (0, 1, or 2).

2. Calculate Sum of Out-Degrees
Iterate through the matrix row-by-row and sum all values.

3. Calculate Sum of In-Degrees
Iterate through the matrix column-by-column and sum all values.

4. Verify the Theorem
Confirm that Sum of Out-Degrees == Sum of In-Degrees.

5. Measure Performance
Record the total execution time from start to finish in milliseconds.



Building the Digital Labyrinth: Random Graph Generation

The foundation of our experiment is a randomly generated adjacency matrix. Here's how we built it.

Seeding for True Randomness

We first call srand(time(NULL)) once. This uses the current time as a unique starting point (a "seed"), ensuring we get a completely different random graph every time the program runs.

Populating the Matrix

- A nested for loop iterates through every cell of the n x n matrix.
- For each cell [i][j], the core logic is rand() % 3;.
  - rand(): Generates a large pseudo-random integer.
  - % 3: The modulo operator divides that integer by 3 and takes the remainder, which is always 0, 1, or 2.

The Code in Action

```
// Seed the random number generator once
srand(time(NULL));

// For every cell in the n x n grid...
for (i = 0; i < n; i++) {
   for (j = 0; j < n; j++) {
        if (i == j) {
            adj_matrix[i][j] = 0;
        } else {
      // Assign a random edge weight of 0, 1, or 2
      adj_matrix[i][j] = rand() % 3;
   }
}
      sum_of_out_degrees += current_out_degree;
}
```

Anatomy of the Algorithm

The core logic uses two structurally identical nested loops. This ensures every edge weight is counted exactly once for the out-degree sum and exactly once for the out-degree and in-degree sum.

Calculating Sum of Out-Degrees

```
sum_of_out_degrees = 0;
for (i = 0; i < n; i++) { // For each vertex...
    int current_out_degree = 0;
    for (j = 0; j < n; j++) {
        // Sum its outgoing edges (Row)
        current_out_degree += adj_matrix[i][j];
    }
    sum_of_out_degrees += current_out_degree;
}
```

Calculating Sum of In-Degrees

```
sum_of_in_degrees = 0;
for (j = 0; j < n; j++) { // For each vertex...
    int current_in_degree = 0;
    for (i = 0; i < n; i++) {
        // Sum its incoming edges (column)
        current_in_degree += adj_matrix[i][j];
    }
    sum_of_in_degrees += current_in_degree;
}
```

Verify the Theorem

Confirm that Sum of Out-Degrees == Sum of In-Degrees

```
if (sum_of_in_degrees == sum_of_out_degrees) {
    printf("✅ Verification successful: Sums are equal.\n");
} else {
    printf("❌ Verification failed: Sums are NOT equal.\n");
}
```

Program Output in Action

Here is a sample of the actual terminal output from our program, showing both the verification for n=1000 and a small snippet of the generated 1000x1000 adjacency matrix.

```
--- Analysis for n = 1000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 998438
Sum of all in-degrees:  998438
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 20.81 ms
```

Verification Output for n = 1000

Data Persistence: Visualizing the Matrix Snippet

Displaying the massive 1000x1000 matrix required a practical approach to avoid unreadable output.

The Challenge: The "One Million Number" Problem

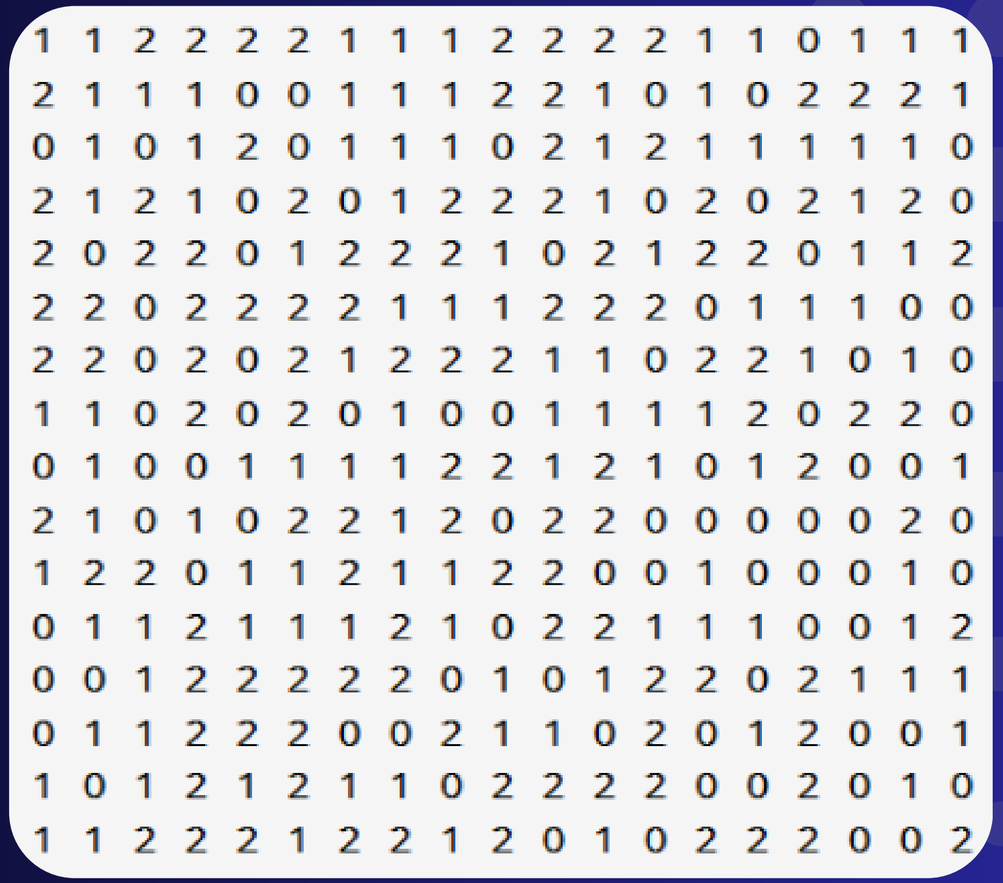
- Directly printing the entire 1000x1000 matrix would flood the console with one million numbers.
- This would make the program's output impossible to read and verify visually.

Our Solution: Print a Representative Snippet

- Instead of printing everything, we decided to display just the top-left corner of the matrix. This proves the data was generated correctly while keeping the output clean.

Program Output in Action

Here is a sample of the actual terminal output from our program, showing both the verification for n=1000 and a small snippet of the generated 1000x1000 adjacency matrix.



1 1 2 2 2 2 1 1 1 2 2 2 2 1 1 0 1 1 1
2 1 1 1 0 0 1 1 1 2 2 1 0 1 0 2 2 2 1
0 1 0 1 2 0 1 1 1 0 2 1 2 1 1 1 1 1 0
2 1 2 1 0 2 0 1 2 2 2 1 0 2 0 2 1 2 0
2 0 2 2 0 1 2 2 2 1 0 2 1 2 2 0 1 1 2
2 2 0 2 2 2 2 1 1 1 2 2 2 0 1 1 1 0 0
2 2 0 2 0 2 1 2 2 2 1 1 0 2 2 1 0 1 0
1 1 0 2 0 2 0 1 0 0 1 1 1 1 2 0 2 2 0
0 1 0 0 1 1 1 1 2 2 1 2 1 0 1 2 0 0 1
2 1 0 1 0 2 2 1 2 0 2 2 0 0 0 0 0 2 0
1 2 2 0 1 1 2 1 1 2 2 0 0 1 0 0 0 1 0
0 1 1 2 1 1 1 2 1 0 2 2 1 1 1 0 0 1 2
0 0 1 2 2 2 2 2 0 1 0 1 2 2 0 2 1 1 1
0 1 1 2 2 2 0 0 2 1 1 0 2 0 1 2 0 0 1
1 0 1 2 1 2 1 1 0 2 2 2 2 0 0 2 0 1 0
1 1 2 2 2 1 2 2 1 2 0 1 0 2 2 2 0 0 2

Adjacency Matrix Snippet (for n = 1000)

Data Persistence: Visualizing the Matrix Snippet

Displaying the massive 1000x1000 matrix required a practical approach to avoid unreadable output.

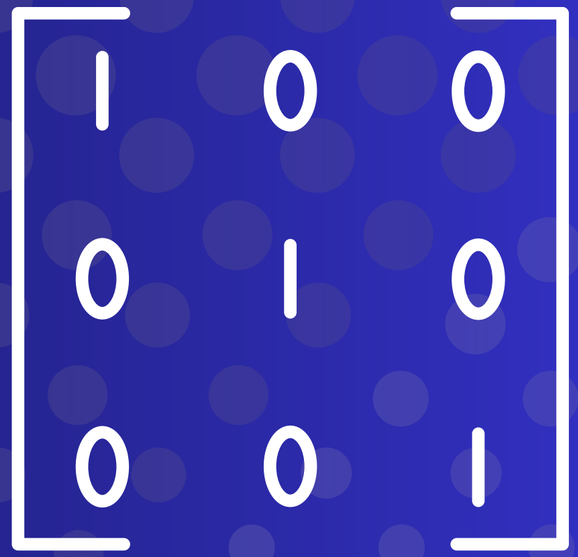
Step 1: Define a Snippet Size

- We added a constant, SNIPPET_SIZE, and set its value to 15. This means we will show a 15x15 portion of the matrix. *#define SNIPPET_SIZE 15*

Step 2: Modify the Final Print Loop

- The final loop in the main function was changed to iterate only up to SNIPPET_SIZE instead of 1000.

```
// In main(), after all other analysis is complete...
printf("\n--- Adjacency Matrix Snippet for n = 1000...---\n");
for (int i = 0; i < SNIPPET_SIZE; i++) {
    for (int j = 0; j < SNIPPET_SIZE; j++) {
        // This now prints only the 15x15 snippet
        printf("%d ", matrix_to_print[i][j]);
    }
    printf("\n");
}
```



Against the Clock: How We Measured Execution Time

To analyze the algorithm's efficiency, we needed a precise way to measure the time taken for computation. We used the clock() function from C's <time.h> library.

The Concept of "Clock Ticks"

- The clock() function doesn't measure seconds directly. Instead, it counts CPU clock ticks—the smallest unit of time for a processor. This provides a very precise measurement of how much work the CPU has actually done.

The Measurement Process

1. Start the Timer: Before the heavy computation begins, we call start = clock(); to record the starting tick count.
2. Run the Algorithm: The graph generation and degree calculations are executed.
3. Stop the Timer: Immediately after, we call end = clock(); to get the final tick count.

Converting Ticks to Milliseconds

- We use the constant CLOCKS_PER_SEC (which tells us how many ticks are in one second on that specific machine) to convert the tick difference into milliseconds.

Against the Clock: How We Measured Execution Time

// 1. Record start time
start = clock();

// ... (All graph generation and degree calculations happen here) ...

// 2. Record end time
end = clock();

// 3. Calculate time in milliseconds

- total_time = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

```
start = clock();

for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (i == j) {
            adj_matrix[i][j] = 0;
        } else {
            adj_matrix[i][j] = rand() % 3;
        }
    }
}

for (i = 0; i < n; i++) {
    int current_out_degree = 0;
    for (j = 0; j < n; j++) {
        current_out_degree += adj_matrix[i][j];
    }
    sum_of_out_degrees += current_out_degree;
}

for (j = 0; j < n; j++) {
    int current_in_degree = 0;
    for (i = 0; i < n; i++) {
        current_in_degree += adj_matrix[i][j];
    }
    sum_of_in_degrees += current_in_degree;
}

end = clock();
```

Results: Theorem Verification

The experiment was a complete success. For every graph size tested, the sum of in-degrees was mathematically identical to the sum of out-degrees, providing powerful computational proof of the theorem.

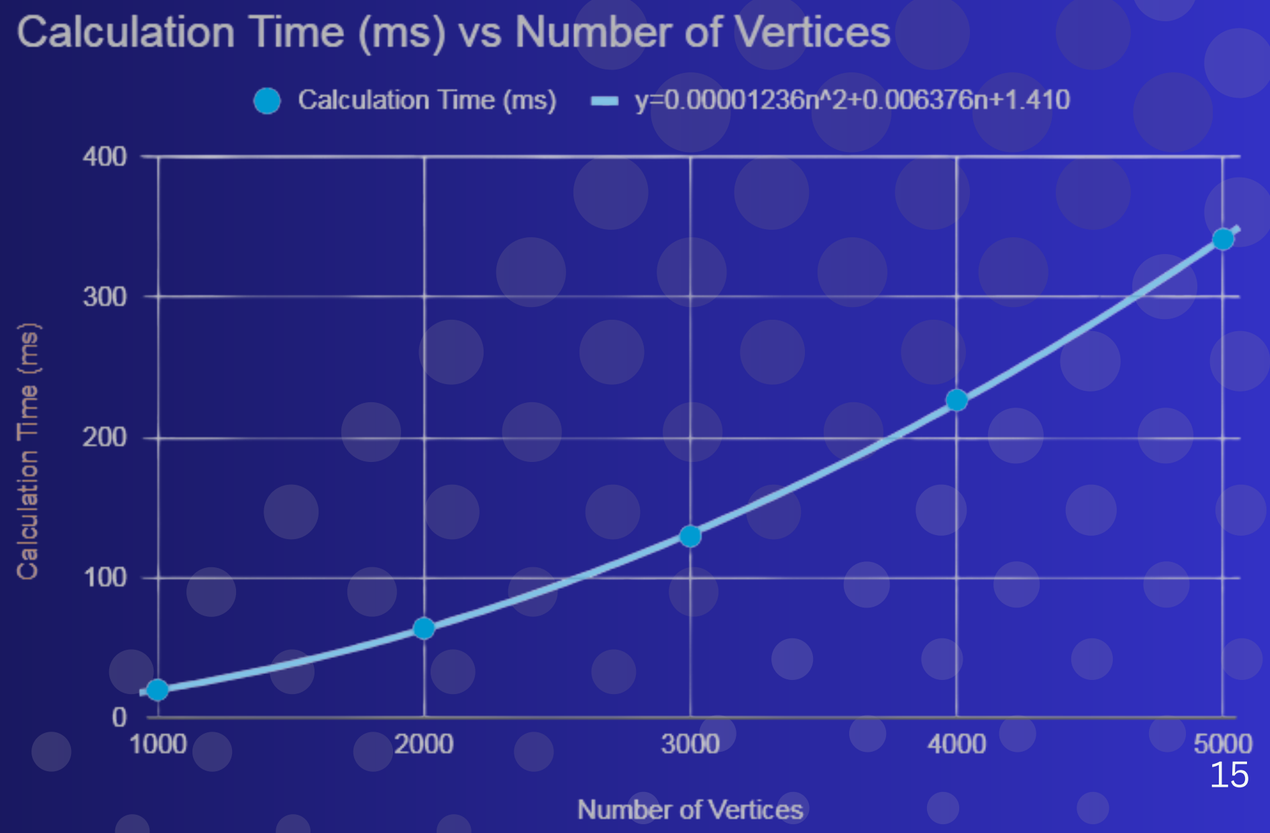
Vertices (n)	Sum of Out-Degrees	Sum of In-Degrees	Verification
1000	998,438	998,438	✅ Success
2000	4,000,185	4,000,185	✅ Success
3000	8,993,886	8,993,886	✅ Success
4000	15,998,677	15,998,677	✅ Success
5000	24,991,350	24,991,350	✅ Success

Performance Analysis Overview

The graph of our timing data shows a clear upward curve, proving the relationship between vertex count and computation time is not linear. As the graph size increases, the time required to analyze it grows at an accelerating rate.
[Chart showing a smooth quadratic curve for "Calculation Time (ms) vs. Number of Vertices"]
This quadratic relationship perfectly matches the theoretical predictions for our algorithm's efficiency.

Vertices (n)	Calc. time (ms)
1000	20.21
2000	64.15
3000	129.75
4000	226.83
5000	341.55

=



Performance Analysis: The Empirical Evidence

The graph of our timing data visually confirms the algorithm's time complexity.

Observation: A Non-Linear Relationship

- The graph shows a distinct upward curve, not a straight line.
- This proves that as the number of vertices (n) increases, the computation time grows at an accelerating rate.

Deriving the Equation from Data

- Because the curve is quadratic, we can model it with a polynomial equation: $Time(n) = an^2 + bn + c$
- By fitting this model to our data points (e.g., (1000, 20.21 ms), (2000, 64.15 ms), etc.), we can solve for the constants a, b, and c.

The resulting equation for our data is approximately: $Time(n) \approx y=0.00001236n^2+0.006376n+1.410$

Conclusion from the Graph

- Using $f(n) <= C \times g(n)$
- In Big O notation, we focus on the fastest-growing term, which is n^2.
- Therefore, the experimental data provides strong empirical evidence that the algorithm has a time complexity of $O(n^2)$.

Time Complexity: The Theoretical Proof

Beyond the data, the algorithm's time complexity is embedded directly in the source code's structure.

The Source of Complexity: Nested Loops

- The runtime is dominated by three main operations, each requiring a full pass through the n x n adjacency matrix.
- Any operation that "touches" every cell in an n x n grid will have a complexity related to n^2.

Analyzing the Code Step-by-Step

1. Graph Generation: A nested for loop runs n times for the rows, and for each row, n times for the columns.
   - Total Operations = n * n = n^2.
2. Out-Degree Sum: A second, independent nested loop sums all rows.
   - Total Operations = n * n = n^2.
3. In-Degree Sum: A third nested loop sums all columns.
   - Total Operations = n * n = n^2.

Final Complexity Calculation

- The total complexity is the sum of the dominant operations: $O(n^2) + O(n^2) + O(n^2) = O(3n^2)$
- In Big O notation, we drop constant factors (like the 3).
- This leaves a final theoretical time complexity of $O(n^2)$, which perfectly matches our experimental findings.

Conclusion Milestones

Theorem Verified

We successfully demonstrated that Sum(in-degrees) = Sum(out-degrees), providing strong computational evidence for a fundamental theorem of graph theory.

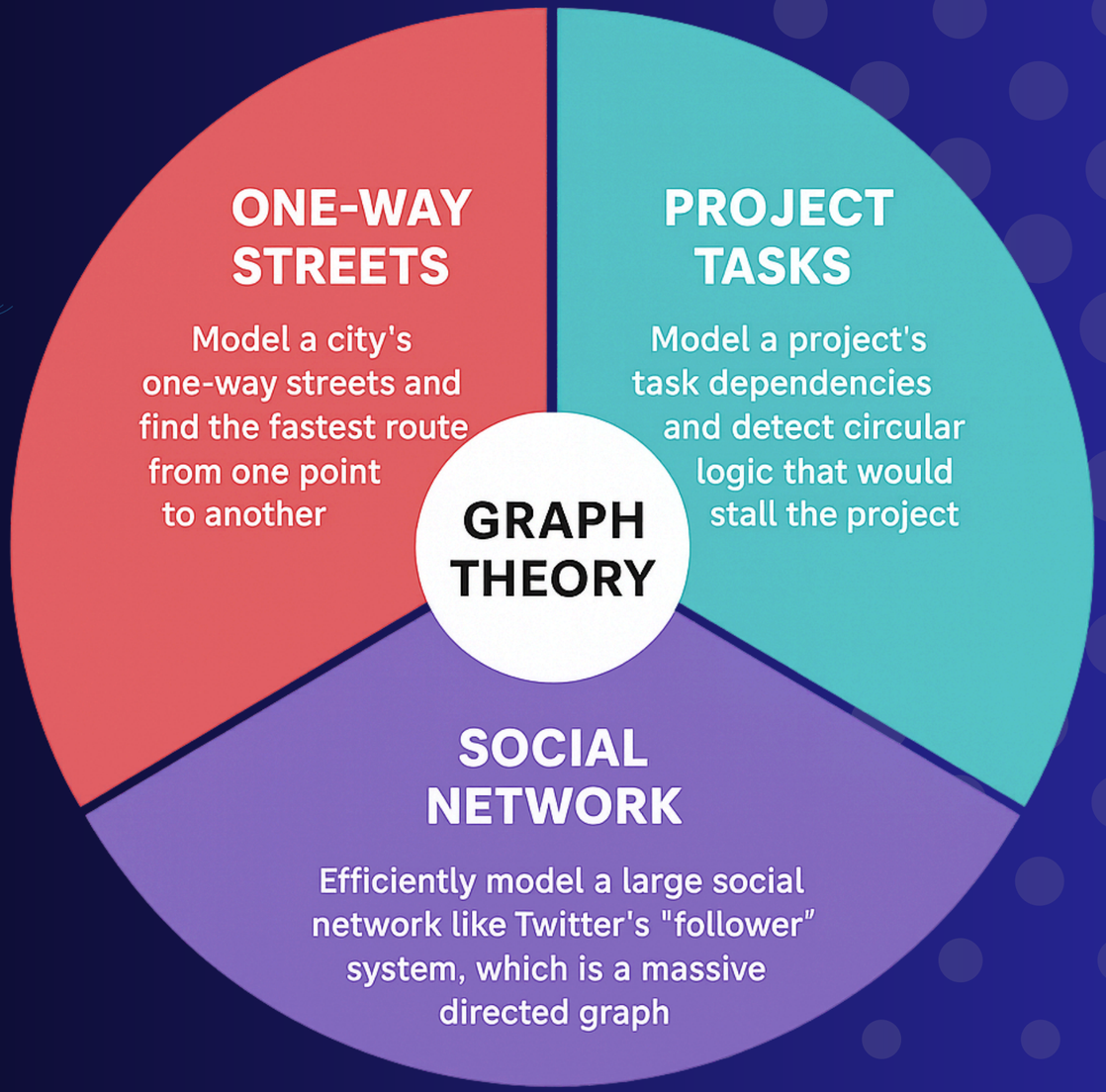
Performance Confirmed

Our empirical timing data produced a textbook $O(n^2)$ performance curve, validating the theoretical time complexity for adjacency matrix operations.

Theory Meets Practice

This project effectively bridged the gap between an abstract mathematical theorem and a practical, real-world code implementation and analysis.

Future Work


ONE-WAY STREETS
Model a city's one-way streets and find the fastest route from one point to another
PROJECT TASKS
Model a project's task dependencies and detect circular logic that would stall the project
GRAPH THEORY
SOCIAL NETWORK
Efficiently model a large social network like Twitter's "follower" system, which is a massive directed graph

THANK YOU!!

pls dont ask 😭